# Prototype Ingestion Pipeline -- Vision Document (v2)

The Prototype Ingestion Pipeline automates the handoff between UI/UX design and production engineering. When the design team finishes a prototype in their dedicated repo, an automated pipeline extracts key metadata -- screenshots, component hierarchies, interaction notes -- and pushes a UI Story to Azure DevOps. From there, developers pull stories into beads and integrate the prototypes into the production micro-frontend. The result: every design-to-code transition is tracked, visible, and contextualized from the moment it leaves the designer's hands.

> This v2 supersedes the original vision document, updated to reflect the implemented pipeline. Changes are minor -- the core vision held through implementation.

---

## The Problem

Translating UI/UX prototypes into production MFE code is manual, context-lossy, and hard to track. Designers build screens in one repo; engineers rebuild them in another. Along the way, intent gets lost -- which components were used, what interactions were intended, what the screen was supposed to look like. There is no automated bridge, no traceable artifact, and no structured handoff.

---

## The Workflow

The pipeline operates in three phases: Design, Ingestion, and Integration.

```
None
flowchart TD
    subgraph phase1 ["Phase 1: Design"]
        A[UI/UX team builds prototypes\nwith Modus Web Components] --> B[Merge
prototype branch\nto main]
    end

    subgraph phase2 ["Phase 2: Ingestion"]
        B -->|"GitHub Action trigger\n(src/pages/** changed)"| D[Detect changed
prototypes\nvia git diff]
        D --> E[AI agent extracts metadata\nand captures screenshots]
        E --> F[Create UI Story\non Azure DevOps board]
```

```
    end

    subgraph phase3 ["Phase 3: Integration"]
        F --> G[Pull ADO story into\nbeads via bd tooling]
        G --> H[Developer integrates prototype\ninto production MFE]
    end
```

## Phase 1 -- Design

The UI/UX team works in the `lista-payroll-design` repository. They build prototypes using Modus Web Components in Cursor, focusing purely on the user experience without concern for production architecture.

Designers work on feature branches and iterate freely. When a prototype is considered complete, the designer merges their branch to main. **Merging to main signals that the prototype is ready for ingestion.** There is no separate folder to move files into -- prototypes live in `src/pages/` exactly where they are built.

## Phase 2 -- Ingestion

A GitHub Action triggers on merge to main when files under `src/pages/` have changed. The pipeline detects which prototype pages were modified, resolves their application routes, and for each one extracts structured metadata:

- **Screenshots** -- rendered captures of the prototype, including individual steps for multi-step flows like wizards
- **Component Hierarchy** -- the tree of Modus and custom components used, with their props
- **Page/Feature Name** -- derived automatically from the file or folder name
- **Interactive Elements** -- buttons, forms, inputs, navigation, and their types

The pipeline then creates a **UI Story** on the Azure DevOps board with this metadata attached. The story represents a unit of design work ready for engineering consideration.

## Phase 3 -- Integration

A developer or lead sees the UI Story appear on the ADO board. Using the **Lista Beads extension** (available on both the VS Code and Cursor marketplaces), they pull the story into beads directly from their IDE. The extension bridges ADO and the local `bd` tooling, so the developer never leaves their editor to pick up work.

The bead carries the context forward: the screenshots, the component hierarchy, and the interaction notes. The developer is not starting from scratch. They use the bead context to integrate the prototype into `prism-ui-react-payroll`, mapping the design's components and interactions to the production architecture.

---

## What Gets Extracted

Each prototype produces four categories of metadata that flow into the ADO story.

| Extraction Target | Method | Result in ADO Story |
|---|---|---|
| **Screenshots** | AI agent driving Playwright via MCP | Image attachments on the story |
| **Component Hierarchy** | AI agent reading source files and following imports | Recursive tree of Modus and custom components with props |
| **Page/Feature Name** | Derived from file/folder name in `src/pages/` | Story title and description |
| **Interactive Elements** | AI agent identifying buttons, inputs, navigation in source | Typed list with labels and positions |

---

## Iteration and Versioning

Prototypes are not always one-and-done. A designer may revise a prototype after it has already been ingested -- whether from design feedback, usability testing, or changing requirements.

### Version Derivation

The pipeline automatically assigns a version number to each prototype it ingests. Before creating a new UI Story, the pipeline queries the ADO board for existing stories with the same prototype identifier. If none exist, the prototype is tagged **v1**. If previous versions are found, the version increments accordingly (**v2**, **v3**, etc.).

The version appears in the story title (e.g., "Prototype: Tax Wizard (v2)") and the new story links back to the previous version via an ADO relation. The designer does not manage versions -- they simply update the prototype and merge. The pipeline handles the rest.

### How Iterations Are Handled

When a previously ingested prototype is modified and merged to main again, the pipeline creates a **new UI Story** rather than updating the original. This preserves a clean audit trail: each version of the prototype exists as its own story on the ADO board, and no developer is surprised mid-integration by silently changing requirements.

| **Scenario** | **What Happens** |
|---|---|
| **Pre-integration** (nobody has started work) | New versioned story is created. The original can be closed or deprioritized on the board. |
| **Mid-integration** (developer is actively working) | New versioned story is created. Developer and lead decide whether to continue with the original, switch to the new version, or reconcile the two. |
| **Post-integration** (already in production) | New versioned story is created as a follow-up. Treated as a design revision to an existing production screen. |

This approach keeps the board honest -- every version of a prototype that was ever ingested is traceable, and the decision of how to handle superseded work stays with the humans, not the pipeline.

---

## Why This Approach

This MVP prioritizes simplicity and traceability over sophistication.

- **Leverages existing tooling.** The ADO-to-beads pipeline already exists. The ingestion pipeline feeds into it rather than replacing it.
- **Adds visibility.** UI Stories on the ADO board give the entire team line-of-sight into incoming design work.
- **Maintains traceability.** Every prototype's journey -- from designer repo to ADO story to bead to production PR -- is tracked.
- **Keeps the pipeline focused.** Extract and push. No SDK analysis, no alignment scoring, no context injection. The developer handles technical integration decisions.
- **Zero designer friction.** Designers do not move files, write metadata, or manage versions. They build and merge -- the pipeline does the rest.
- **Clean separation of concerns.** The pipeline handles logistics; the developer handles architecture.

---

## Terminology

| Term | Meaning |
|---|---|
| **Prototype** | A UI/UX screen built by the design team using Modus Web Components in their dedicated repo. |
| **Slug** | A URL-safe identifier for a prototype, derived from the page file or folder name in `src/pages/`. |
| **Ingestion** | The automated process of detecting changed prototypes, extracting metadata, and creating ADO stories. |
| **Integrate** | The developer's work of taking a prototype's design and wiring it into the production MFE architecture. |
| **UI Story** | The Azure DevOps work item created by the pipeline, carrying the prototype's metadata and screenshots. |
| **Bead** | The local developer unit of work, created by pulling a UI Story from ADO into the `bd` tooling. |